Guichês de passes no metrô de Paris: documentação da letra (b)

Visão geral

A letra (b) pede o menor número de guichês de modo que nenhum turista caminhe mais que uma estação até um guichê, e isso é exatamente o problema do **Conjunto Dominante Mínimo**. O arquivo `guiches.py` resolve esse problema de três formas: força bruta, branch-and-bound e heurística gulosa.

Modelagem. O mapa é um grafo não direcionado: cada vértice é uma estação e cada aresta é um trecho entre duas estações consecutivas de uma linha. Um conjunto S de estações com guichê é válido quando toda estação está em S ou é vizinha de uma estação de S. O objetivo é minimizar o tamanho de S.

Por que três métodos. O Conjunto Dominante Mínimo é NP-difícil, então não se conhece algoritmo exato rápido para todos os casos. Por isso o trabalho pede um método exato simples (força bruta), um método exato mais esperto (branch-and-bound) e uma aproximação em tempo polinomial (heurística).

O que o arquivo entrega:

- `forca_bruta`, `branch_and_bound` e `heuristica_gulosa`, cada uma devolvendo as estações escolhidas, a quantidade, o tempo e o número de nós ou testes.
- `resolver(g, metodo)`, uma porta de entrada única para a interface gráfica.
- `comparar(g)`, que roda os três métodos, confere se cada solução realmente domina o grafo e imprime a comparação.
- Funções para ler os arquivos de estações e de linhas no formato do enunciado.
- Um gerador de grafos sintéticos, usado para testar sem o mapa real.

Representação do grafo

Cada estação vira um índice `i`, e sua **vizinhança fechada** N[i] (a própria estação mais as vizinhas) é guardada como um número inteiro em que cada bit representa uma estação. Com isso, verificar se um conjunto de guichês cobre o mapa inteiro vira uma sequência de operações OU entre inteiros.

- `viz[i]` é a máscara de N[i].
- `todos` é a máscara com os n bits ligados, ou seja, o conjunto de todas as estações.
- Um conjunto S domina o grafo quando o OU das máscaras `viz[v]`, para v em S, é igual a `todos`.

- `bit_count()` conta quantas estações uma máscara cobre, o que permite medir o ganho de cada escolha rapidamente.

A classe `Grafo` recebe a lista de nomes e a lista de pares de estações conectadas. Ela remove nomes duplicados, ignora arestas com estações desconhecidas e arestas de uma estação com ela mesma, e monta `adj` (conjuntos de vizinhos) e `viz` (máscaras). O método `domina(conjunto)` faz o teste descrito acima.

Os resultados voltam em um objeto `Resultado` com: nome do método, índices e nomes das estações escolhidas, tempo em segundos, número de nós visitados (ou subconjuntos testados) e a marca `completo`, que vale falso quando o limite de tempo estourou.

Força bruta

A força bruta testa subconjuntos de estações em ordem crescente de tamanho e devolve o primeiro que cobre o mapa inteiro, que por isso é garantidamente de tamanho mínimo.

```
FORCA_BRUTA(G):
  para k de 1 até n:
    para cada subconjunto S de V com |S| = k:
      se a união das N[v], v em S, cobre V:
        retorne S
```

Complexidade. No pior caso testa todos os subconjuntos de tamanho até o ótimo, ou seja, o número de testes é a soma de C(n, k) para k de 1 até OPT, cada teste custando k operações OU. Isso é exponencial em n: com 25 estações foram 856.231 testes, e com 37 estações o programa já não terminou em 20 segundos. Em memória usa apenas o subconjunto atual, O(n).

Limite de tempo. Se o parâmetro `limite_tempo` estoura, a função para, marca o resultado como incompleto e devolve a solução gulosa só para haver uma resposta útil. Esse resultado aparece como "(tempo esgotado)" e não deve ser tratado como ótimo.

Branch-and-bound

O branch-and-bound encontra o mesmo ótimo da força bruta, mas descarta ramos da árvore de busca que não podem melhorar a melhor solução já conhecida.

Ramificação. Escolhe-se uma estação v ainda não coberta. Alguma estação de N[v] precisa receber guichê, então a busca ramifica sobre esses candidatos, começando pela estação v que tem menos candidatos livres. Depois de explorar o ramo em que o candidato c entra, c fica proibido nos ramos irmãos seguintes, o que evita gerar a mesma solução duas vezes. Os candidatos são tentados em ordem decrescente de ganho.

Limite superior. É o tamanho da melhor solução conhecida. Começa com a solução da heurística gulosa e melhora sempre que a busca encontra algo menor.

Limite inferior. Se já há |S| guichês escolhidos e faltam F estações descobertas, e o melhor ganho possível de uma única escolha é G estações novas, então ainda são necessários pelo menos ⌈F / G⌉ guichês. O limite é |S| + ⌈F / G⌉.

Poda. Se o limite inferior é maior ou igual ao limite superior, o ramo não pode vencer a melhor solução e é abandonado. O ramo também é abandonado quando nenhuma estação permitida cobre algo novo.

```
BNB(G):
  melhor ← GULOSO(G)                    // limite superior inicial
  REC(dominado = ∅, S = [], proibido = ∅)
  retorne melhor

REC(dominado, S, proibido):
  se dominado = V:
    se |S| < |melhor|: melhor ← S
    retorne
  faltam ← V \ dominado
  G ← máximo, entre v fora de proibido, de |N[v] ∩ faltam|
  se G = 0: retorne                     // beco sem saída
  LB ← |S| + ⌈|faltam| / G⌉
  se LB ≥ |melhor|: retorne             // poda
  v ← estação de faltam com menos candidatos em N[v] \ proibido
  para cada c em N[v] \ proibido, em ordem decrescente de ganho:
    REC(dominado ∪ N[c], S + [c], proibido)
    proibido ← proibido ∪ {c}
```

Por que está correto. Toda solução precisa conter alguma estação de N[v], e proibir os candidatos já explorados divide o espaço de soluções sem repetição nem perda. O limite inferior é válido porque cada guichê novo cobre no máximo G estações ainda descobertas.

Efeito prático. Nos testes, o branch-and-bound visitou 17 nós onde a força bruta testou 5.760 subconjuntos (15 estações), e 47 nós contra 856.231 testes (25 estações). O pior caso continua exponencial, e com 131 estações ele não terminou em 20 segundos.

Heurística gulosa

A heurística escolhe, a cada passo, a estação que cobre mais estações ainda descobertas, e roda em tempo polinomial mesmo no pior caso, sem garantir o ótimo.

```
GULOSO(G):
  descoberto ← V;  S ← []
  enquanto descoberto ≠ ∅:
    v ← vértice que maximiza |N[v] ∩ descoberto|
    S ← S + [v];  descoberto ← descoberto \ N[v]
  para cada v em S:                     // limpeza de guichês redundantes
    se S \ {v} ainda domina V: remova v de S
  retorne S
```

Complexidade. O laço principal faz no máximo n iterações, e cada uma examina os n vértices com uma operação E e uma contagem de bits sobre máscaras. A limpeza testa cada guichê contra os demais. O total é $O(n^2)$ operações de máscara, portanto polinomial. A memória é O(n).

Garantia de aproximação. Conjunto Dominante é um caso do problema de Cobertura de Conjuntos, em que os conjuntos são as vizinhanças fechadas N[v]. O guloso clássico dessa cobertura devolve uma solução de tamanho no máximo $H(\Delta+1)$ vezes o ótimo, com H o número harmônico e Δ o grau máximo do grafo. Como $H(\Delta+1) \leq 1 + \ln(\Delta+1)$, a heurística nunca fica muito longe do ótimo, e em metrôs Δ é pequeno (as estações de baldeação ligam poucas linhas).

Por que essa heurística.

- É simples de explicar e de implementar com as mesmas máscaras já usadas nos métodos exatos.
- É polinomial e tem garantia teórica conhecida, o que atende ao pedido de justificar a escolha.
- Nos testes ela igualou o ótimo em todos os grafos em que o ótimo foi calculado (15, 25 e 37 estações).
- Ela também fornece o limite superior inicial do branch-and-bound, o que acelera a poda desde o primeiro nó.

Implementação

Tudo está em um único arquivo Python, `guiches.py`, que usa só `time`, `itertools` e `dataclasses` da biblioteca padrão, então atende à regra de não depender de bibliotecas fora das instaladas no laboratório. Ele precisa de **Python 3.10 ou mais novo**, porque usa `int.bit_count()`.

Leitura dos arquivos.

- `ler_estacoes(caminho)` lê linhas no formato `Nome x y`. O nome pode ter espaços e acentos, porque os dois últimos itens da linha são sempre as coordenadas.
- `ler_linhas(caminho, conhecidas)` lê seções que começam com `LinhaN cor k`, seguidas de k linhas `Nome1 Nome2`. Como os nomes podem ter espaços, a função separa o par testando as divisões possíveis contra a lista de estações conhecidas; se a linha tiver tabulação, `;` ou `|`, usa esse separador.
- `grafo_de_arquivos(arq_estacoes, arq_linhas, linhas_usadas=None)` monta o `Grafo`. O parâmetro `linhas_usadas`, por exemplo `{"Linha1", "Linha4"}`, permite testar com menos linhas, como o enunciado pede.

Execução pela linha de comando.

```
python guiches.py                    # demonstração com grafos sintéticos
```

python guiches.py estacoes.txt linhas.txt   # roda os 3 métodos no mapa do grupo (limite de 60 s)

Integração com a interface gráfica. A interface só precisa de uma chamada:

```
from guiches import grafo_de_arquivos, resolver

g = grafo_de_arquivos("estacoes.txt", "linhas.txt")
r = resolver(g, "branch_and_bound", limite_tempo=60)
# r.estacoes -> nomes das estações com guichê (para pintar no mapa)
# r.quantidade, r.tempo, r.nos_visitados, r.completo
```

Os valores aceitos para o método são `"forca_bruta"`, `"branch_and_bound"` e `"heuristica"`. Se `r.completo` for falso, a interface deve avisar que o tempo esgotou e que a solução mostrada não é garantidamente ótima.

Pontos de eficiência. As máscaras de bits deixam união e contagem de cobertura muito baratas, o branch-and-bound começa com um limite superior bom (a solução gulosa) e escolhe sempre a estação mais restrita para ramificar, e o parâmetro `limite_tempo` impede que a interface trave.

Testes e resultados

Nos testes feitos, os três métodos devolveram soluções válidas e os dois métodos exatos sempre concordaram com o ótimo.

Validação contra um verificador independente. Foram gerados 300 grafos aleatórios com 2 a 11 estações. Para cada um, um verificador escrito separadamente calculou o ótimo por busca exaustiva. A força bruta e o branch-and-bound acertaram o ótimo nos 300 casos (0 falhas), a heurística nunca ficou abaixo do ótimo, e toda solução devolvida realmente cobria o grafo.

Leitura de arquivos. Um exemplo com 5 estações, incluindo nomes com espaço e acento ("Gare de Lyon", "République", "Châtelet"), foi lido corretamente, e o filtro por linhas montou o grafo só com a linha pedida.

Tempos em grafos sintéticos. São grafos feitos de linhas em forma de caminho que se cruzam, não o mapa de Paris. O limite de tempo foi de 20 segundos por método, e os tempos dependem da máquina.

Grafo sintético	Força bruta	Branch-and-bound	Heurística gulosa
15 estações (2 linhas)	6 guichês, 0,0012 s, 5.760 testes	6 guichês, 0,0001 s, 17 nós	6 guichês, menos de 0,0001 s

25 estações (3 linhas)	8 guichês, 0,22 s, 856.231 testes	8 guichês, 0,0003 s, 47 nós	8 guichês, menos de 0,0001 s
37 estações (4 linhas)	não terminou em 20 s	13 guichês, 0,0033 s, 431 nós	13 guichês, menos de 0,0001 s
131 estações (10 linhas)	não terminou em 20 s	não terminou em 20 s	43 guichês, 0,0004 s

Na linha de 131 estações o ótimo não é conhecido, então só se sabe que 43 guichês bastam. Nas outras linhas em que o ótimo foi calculado, a heurística o igualou.

Limitações e próximos passos

O algoritmo está validado, mas ainda faltam testes com o mapa real e com a interface gráfica, e o branch-and-bound pode não terminar com muitas linhas.

O que ainda não foi testado:

- O mapa real do grupo (linhas sorteadas, N = 1), porque o arquivo de linhas ainda não existe.
- A integração com a interface gráfica; só a função `resolver()` foi exercitada.
- A medição de tempos na máquina em que o trabalho será apresentado.

Limitações conhecidas:

- Com 131 estações o branch-and-bound estourou 20 segundos. Com as 10 linhas do grupo é provável que ele também não termine, e nesse caso o programa mostra a solução gulosa marcada como "(tempo esgotado)".
- Estações que aparecem no arquivo de coordenadas, mas em nenhum trecho do arquivo de linhas, não entram no grafo.
- Para o relatório, as comparações entre força bruta e branch-and-bound devem usar casos pequenos (poucas linhas), em que os dois terminam.

Como melhorar o branch-and-bound:

- Regra de redução para estações terminais: se uma estação tem um único vizinho, esse vizinho pode ser colocado na solução sem perder o ótimo.
- Resolver separadamente cada parte desconexa do grafo e somar os resultados.
- Um limite inferior mais forte, por exemplo contando estações que não podem ser cobertas pela mesma escolha.